

Configuration

As a starting point for using Spring Data Envers, you need a project with Spring Data JPA on the class-path and an additional `spring-data-envers` dependency:

```
<dependencies>

<!-- other dependency elements omitted -->

<dependency>
  <groupId>org.springframework.data</groupId>
  <artifactId>spring-data-envers</artifactId>
  <version>4.0.2</version>
</dependency>

</dependencies>
```

XML

This also brings `hibernate-envers` into the project as a transient dependency.

To enable Spring Data Envers and Spring Data JPA, we need to configure two beans and a special `repositoryFactoryBeanClass`:

```
@Configuration
@EnableEnversRepositories
@EnableTransactionManagement
public class EnversDemoConfiguration {

    @Bean
    public DataSource dataSource() {

        EmbeddedDatabaseBuilder builder = new EmbeddedDatabaseBuilder();
        return builder.setType(EmbeddedDatabaseType.HSQL).build();
    }

    @Bean
    public LocalContainerEntityManagerFactoryBean entityManagerFactory() {
```

JAVA

```

        HibernateJpaVendorAdapter vendorAdapter = new HibernateJpaVendorAdapter();
        vendorAdapter.setGenerateDdl(true);

        LocalContainerEntityManagerFactoryBean factory = new
LocalContainerEntityManagerFactoryBean();
        factory.setJpaVendorAdapter(vendorAdapter);
        factory.setPackagesToScan("example.springdata.jpa.envers");
        factory.setDataSource(dataSource());
        return factory;
    }

    @Bean
    public PlatformTransactionManager transactionManager(EntityManagerFactory
entityManagerFactory) {

        JpaTransactionManager txManager = new JpaTransactionManager();
        txManager.setEntityManagerFactory(entityManagerFactory);
        return txManager;
    }
}

```

To actually use Spring Data Envers, make one or more repositories into a [RevisionRepository](#) by adding it as an extended interface:

```

interface PersonRepository
    extends CrudRepository<Person, Long>,
        RevisionRepository<Person, Long, Long> 1
{
}

```

JAVA

- ¹ The first type parameter (Person) denotes the entity type, the second (Long) denotes the type of the id property, and the last one (Long) is the type of the revision number. For Envers in default configuration, the revision number parameter should be Integer or Long .

The entity for that repository must be an entity with Envers auditing enabled (that is, it must have an `@Audited` annotation):

```

@Entity
@Audited
class Person {

    @Id @GeneratedValue
    Long id;
}

```

JAVA

```
String name;  
@Version Long version;  
}
```



Copyright © 2005 - 2026 Broadcom. All Rights Reserved. The term "Broadcom" refers to Broadcom Inc. and/or its subsidiaries.

[Terms of Use](#) • [Privacy](#) • [Trademark Guidelines](#) • [Thank you](#) • [Your California Privacy Rights](#) • [Cookies Settings](#)

Apache®, Apache Tomcat®, Apache Kafka®, Apache Cassandra™, and Apache Geode™ are trademarks or registered trademarks of the Apache Software Foundation in the United States and/or other countries. Java™, Java™ SE, Java™ EE, and OpenJDK™ are trademarks of Oracle and/or its affiliates. Kubernetes® is a registered trademark of the Linux Foundation in the United States and other countries. Linux® is the registered trademark of Linus Torvalds in the United States and other countries. Windows® and Microsoft® Azure are registered trademarks of Microsoft Corporation. "AWS" and "Amazon Web Services" are trademarks or registered trademarks of Amazon.com Inc. or its affiliates. All other trademarks and copyrights are property of their respective owners and are only mentioned for informative purposes. Other names may be trademarks of their respective owners.